



Minimum Depth of a Binary Tree (easy)

We'll cover the following ^

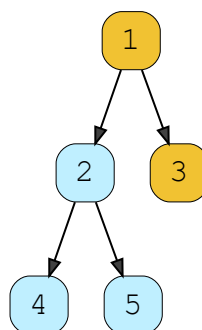
- Problem Statement
- Try it yourself
- Solution
- Code
- Time complexity
- Space complexity
- Similar Problems

Problem Statement

Find the minimum depth of a binary tree. The minimum depth is the number of nodes along the **shortest path from the root node to the nearest leaf node**.

Example 1:

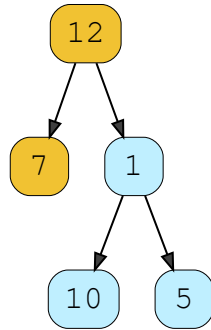
Minimum Depth: 2



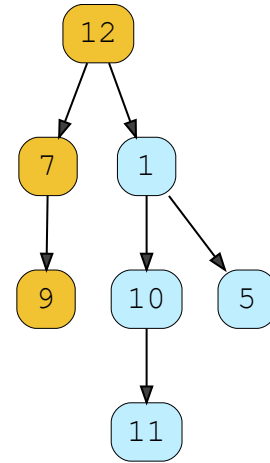
Example 2:



Minimum Depth: 2



Minimum Depth: 3



Try it yourself

Try solving this question here:



Java



Python3



JS



C++

```

1  import java.util.*;
2
3  class TreeNode {
4      int val;
5      TreeNode left;
6      TreeNode right;
7
8      TreeNode(int x) {
9          val = x;
10     }
11 };
12
13 class MinimumBinaryTreeDepth {
14     public static int findDepth(TreeNode root) {
15         // TODO: Write your code here
16         return -1;
17     }
18
19     public static void main(String[] args) {
20         TreeNode root = new TreeNode(12);
21         root.left = new TreeNode(7);
22         root.right = new TreeNode(1);
23         root.right.left = new TreeNode(10);
24         root.right.right = new TreeNode(5);
25         System.out.println("Tree Minimum Depth: " + MinimumBinaryTreeDepth.findDepth(root));
26         root.left.left = new TreeNode(9);
27         root.right.left.left = new TreeNode(11);
28         System.out.println("Tree Minimum Depth: " + MinimumBinaryTreeDepth.findDepth(root));
29     }
30 }
31

```

Run

Save

Reset



Solution

This problem follows the [Binary Tree Level Order Traversal](#) pattern. We can follow the same **BFS** approach. The only difference will be, instead of keeping track of all the nodes in a level, we will only track the depth of the tree. As soon as we find our first leaf node, that level will represent the minimum depth of the tree.

Code

Here is what our algorithm will look like, only the highlighted lines have changed:

 Java

 Python3

 C++

 JS

```
1  import java.util.*;
2
3  class TreeNode {
4      int val;
5      TreeNode left;
6      TreeNode right;
7
8      TreeNode(int x) {
9          val = x;
10     }
11 };
12
13 class MinimumBinaryTreeDepth {
14     public static int findDepth(TreeNode root) {
15         if (root == null)
16             return 0;
17
18         Queue<TreeNode> queue = new LinkedList<>();
19         queue.add(root);
20         int minimumTreeDepth = 0;
21         while (!queue.isEmpty()) {
22             minimumTreeDepth++;
23             int levelSize = queue.size();
24             for (int i = 0; i < levelSize; i++) {
25                 TreeNode currentNode = queue.poll();
26
27                 // check if this is a leaf node
28                 if (currentNode.left == null && currentNode.right == null)
29                     return minimumTreeDepth;
30
31                 // insert the children of current node in the queue
32                 if (currentNode.left != null)
33                     queue.add(currentNode.left);
34                 if (currentNode.right != null)
35                     queue.add(currentNode.right);
36             }
37         }
38         return minimumTreeDepth;
39     }
40
41     public static void main(String[] args) {
42         TreeNode root = new TreeNode(12);
43         root.left = new TreeNode(7);
44         root.right = new TreeNode(1);
45         root.right.left = new TreeNode(10);
46         root.right.right = new TreeNode(5);
47         System.out.println("Tree Minimum Depth: " + MinimumBinaryTreeDepth.findDepth(root));
48         root.left.left = new TreeNode(9);
49         root.right.left.left = new TreeNode(11);
50         System.out.println("Tree Minimum Depth: " + MinimumBinaryTreeDepth.findDepth(root));
51     }
52 }
53
```

Run

Save

Reset



Time complexity

The time complexity of the above algorithm is $O(N)$, where 'N' is the total number of nodes in the tree. This is due to the fact that we traverse each node once.

Space complexity

The space complexity of the above algorithm will be $O(N)$ which is required for the queue. Since we can have a maximum of $N/2$ nodes at any level (this could happen only at the lowest level), therefore we will need $O(N)$ space to store them in the queue.

Similar Problems

Problem 1: Given a binary tree, find its maximum depth (or height).

Solution: We will follow a similar approach. Instead of returning as soon as we find a leaf node, we will keep traversing for all the levels, incrementing `maximumDepth` each time we complete a level. Here is what the code will look like:

Java

Python3

C++

JS

```
1 import java.util.*;
2
3 class TreeNode {
4     int val;
5     TreeNode left;
6     TreeNode right;
7
8     TreeNode(int x) {
9         val = x;
10    }
11 };
12
13 class MaximumBinaryTreeDepth {
14     public static int findDepth(TreeNode root) {
15         if (root == null)
16             return 0;
17
18         Queue<TreeNode> queue = new LinkedList<>();
19         queue.add(root);
20         int maximumTreeDepth = 0;
21         while (!queue.isEmpty()) {
22             maximumTreeDepth++;
23             int levelSize = queue.size();
24             for (int i = 0; i < levelSize; i++) {
25                 TreeNode currentNode = queue.poll();
26                 // insert the children of current node in the queue
27                 if (currentNode.left != null)
28                     queue.add(currentNode.left);
29                 if (currentNode.right != null)
30                     queue.add(currentNode.right);
31             }
32         }
33         return maximumTreeDepth;
34     }
35 }
```



```
34     return maximumTreeDepth;  
35 }  
36  
37 public static void main(String[] args) {  
38     TreeNode root = new TreeNode(12);  
39     root.left = new TreeNode(7);  
40     root.right = new TreeNode(1);  
41     root.right.left = new TreeNode(10);  
42     root.right.right = new TreeNode(5);  
43     System.out.println("Tree Maximum Depth: " + MaximumBinaryTreeDepth.findDepth(root));  
44     root.left.left = new TreeNode(9);  
45     root.right.left.left = new TreeNode(11);  
46     System.out.println("Tree Maximum Depth: " + MaximumBinaryTreeDepth.findDepth(root));  
47 }  
48 }  
49
```

RunSaveReset